

PathFinder — Developer Guide

1. Project Structure Overview

PathFinder is a monorepo with two main directories: `frontend/` (React/Vite) and `backend/` (Node/Express). Both use TypeScript.

```
PathFinder/
├── frontend/                                # React SPA
│   ├── src/
│   │   ├── components/                    # Reusable components (DashboardLayout, ProtectedRoute)
│   │   ├── context/                      # React Context providers (Auth, Theme)
│   │   ├── pages/                        # 20 page-level components
│   │   ├── App.tsx                       # Route definitions and ProtectedRoute wrappers
│   │   ├── main.tsx                     # Application entry point
│   │   └── index.css                     # Complete design system
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
└── backend/                               # Express REST API
    ├── middleware/auth.ts                # JWT authentication middleware
    ├── lib/prisma.ts                     # Prisma client singleton
    ├── lib/notifications.ts              # Notification helper functions
    ├── routes/                           # 11 modular route files
    ├── prisma/schema.prisma              # Database schema
    ├── uploads/                          # File upload directory
    ├── index.ts                          # Server entry point
    ├── package.json
    └── tsconfig.json
```

2. Environment Setup

Prerequisites

- Node.js v18 or higher
- npm v9 or higher
- Git

Step-by-Step Setup

Clone the repository:

```
git clone https://github.com/VaibhavJain2906/PathFinder.git
cd PathFinder
```

Backend setup (Terminal 1):

```
cd backend
npm install
```

Create a `.env` file in the backend directory:

```
PORT=5000
DATABASE_URL="file:./dev.db"
JWT_SECRET="<generate-a-secure-random-string>"
GEMINI_API_KEY="<optional-google-ai-api-key>"
```

Initialize the database and start the server:

```
npx prisma generate
npx prisma db push
npm run dev
```

The backend will be running at `http://localhost:5000`.

Frontend setup (Terminal 2):

```
cd frontend
npm install
npm run dev
```

The frontend will be running at `http://localhost:5173`.

3. Coding Standards

TypeScript

- All files use `.ts` (backend) or `.tsx` (frontend) extensions.
- Interfaces are used for API response types and component props.
- The `AuthRequest` interface extends Express's `Request` to include decoded JWT user data.

CSS

- All styles use CSS custom properties defined in `index.css`.
- No inline styles or external CSS frameworks.
- Class names follow BEM-like conventions.

API Design

- All endpoints follow RESTful conventions.
- Request validation is handled at the route level.
- Error responses follow a consistent format: `{ "error": "message" }`.

4. How to Add a New Feature

Adding a New Backend Route Module

1. Create a new file in `backend/routes/` (e.g., `events.ts`).
2. Define a Router, add your endpoints, and export it.
3. Import and mount it in `backend/index.ts`:

```
import eventRoutes from './routes/events';
app.use('/api/events', eventRoutes);
```

4. If the route requires authentication, use the `authenticate` middleware.
5. If the route is role-restricted, add `requireRole('STUDENT')`.

Adding a New Database Table

1. Add the model definition to `backend/prisma/schema.prisma`.
2. Run `npx prisma db push` to apply changes to the database.
3. Run `npx prisma generate` to update the Prisma client TypeScript types.
4. Use the new model in your route handlers via `prisma.newModel.create()` etc.

Adding a New Frontend Page

1. Create a new component in `frontend/src/pages/` (e.g., `Events.tsx`).
2. If the page is behind authentication, wrap the return in `<DashboardLayout>`.
3. Add a route in `frontend/src/App.tsx`:

```
<Route path="/events" element={<ProtectedRoute><Events /></ProtectedRoute>} />
```

4. Add a sidebar link in `DashboardLayout.tsx`.

Adding a New Sidebar Item

1. Open `frontend/src/components/DashboardLayout.tsx`.
2. Import the desired Lucide icon.
3. Add a new entry to the `menuItems` array with the appropriate path, label, icon, and role filter.

5. Database Operations

All database operations go through Prisma. Common patterns:

```
// Create
const user = await prisma.user.create({ data: { email, password, role } });

// Read (with relations)
const profile = await prisma.studentProfile.findUnique({
  where: { userId },
  include: { skills: true, certifications: true }
});

// Update
await prisma.application.update({
  where: { id: applicationId },
  data: { status: 'ACCEPTED' }
});

// Delete (cascades to related records)
await prisma.user.delete({ where: { id: userId } });

// Count
const total = await prisma.opportunity.count({ where: { status: 'PUBLISHED' } });
```

6. Authentication Flow (For Developers)

1. User sends credentials to POST /api/auth/login.
2. Backend validates credentials with `bcrypt.compare()`.
3. Backend signs a JWT: `jwt.sign({ userId, role }, JWT_SECRET, { expiresIn: '1d' })`.
4. Frontend stores the token in `localStorage` via `AuthContext`.
5. Every subsequent API call includes `Authorization: Bearer <token>` header.
6. Backend `authenticate` middleware calls `jwt.verify()` and attaches decoded data to `req.user`.
7. `requireRole()` middleware checks `req.user.role` against the required role.

7. Deployment

Backend (Render)

- **Build Command:** `npm install && npx prisma generate && npx prisma db push`
- **Start Command:** `npx tsx index.ts`
- **Environment Variables:** `PORT`, `DATABASE_URL`, `JWT_SECRET`, `GEMINI_API_KEY`

Frontend (Vercel)

- **Root Directory:** `frontend`
- **Build Command:** `vite build` (override default to `skip tsc`)
- **Environment Variables:** `VITE_API_URL` (set to Render backend URL)

Auto-Deploy

Both Vercel and Render are connected to the GitHub repository's main branch. Any push to main triggers automatic redeployment.

End of Developer Guide